

Practical Work 4: Word Count

Group 16

December 16, 2025

1 Overview

The objective of this lab is to implement the **Word Count** algorithm using the MapReduce paradigm. This serves as a fundamental example to understand how large datasets are processed in distributed systems by counting the frequency of every word in a text corpus.

2 Implementation Strategy

We decided to implement a custom MapReduce framework using **Python's Multiprocessing** module.

2.1 Why this implementation?

Instead of a simple sequential simulation, we chose **Multiprocessing** to mimic a real cluster environment:

- **Parallelism:** Each Mapper runs in a separate system process (Worker), utilizing different CPU cores. This demonstrates the "Scale-out" philosophy of MapReduce.
- **Independence:** Each worker processes its data chunk independently without sharing memory, enforcing the strict functional programming constraints of MapReduce.
- **Scalability:** The number of workers can be adjusted to match the hardware capabilities (e.g., 2, 4, or 8 workers).

3 System Architecture

3.1 Data Flow Diagram

The following figure illustrates the horizontal data flow from raw text to the final aggregate count.

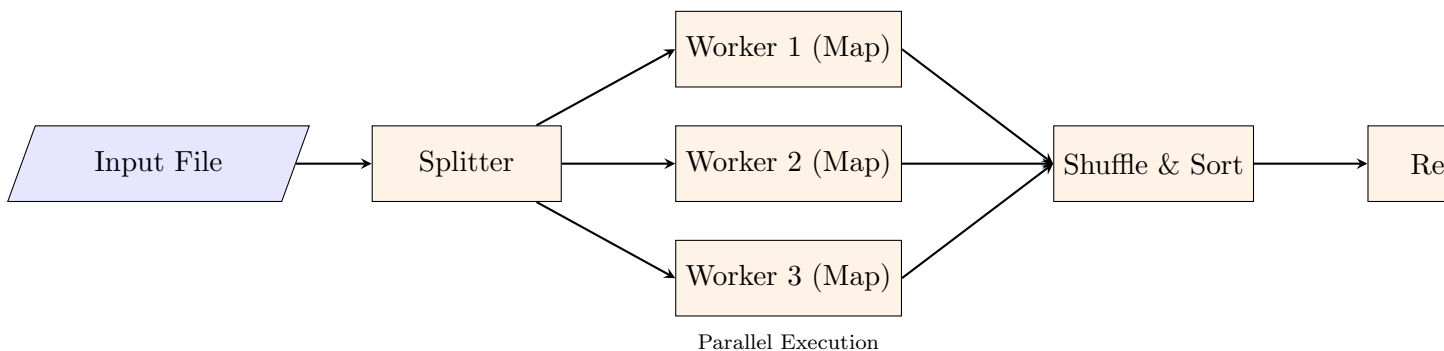


Figure 1: MapReduce Execution Flow

3.2 Mapper Logic

The mapper uses Regular Expressions ('re' module) to cleanly tokenize words, handling punctuation and case sensitivity automatically.

```
1 def map_worker(text_chunk):
2     # Find all words using regex boundary
3     words = re.findall(r'\b\w+\b', text_chunk.lower())
4     return [(word, 1) for word in words]
```

Listing 1: Mapper Worker

3.3 Reducer Logic

The reducer is a simple aggregation function that sums the list of ones generated by the mappers.

```
1 def reduce_worker(item):
2     word, counts = item
3     # Summing the occurrences
4     return (word, sum(counts))
```

Listing 2: Reducer Logic

4 Task Allocation

Member	Role	Responsibilities
Member 1	Core Dev	Implementing Multiprocessing Pool and Map logic.
Member 2	Data Dev	Implementing Shuffle/Sort logic and File I/O handling.
Member 3	Reporter	Designing the LaTeX report and TikZ Architecture diagram.

Table 1: Team Responsibilities